

TECHNICAL EXPLAINER

Sustaining 99.95% Availability Across 25+ Microservices on Oracle Kubernetes Engine

A plain-English guide to the platform: what every component does, how the workflows run end to end, and how the architecture fits together.

The one-line version being explained

“Sustained 99.95% availability across 25+ containerized microservices during releases and traffic spikes by building durable platform primitives on Oracle Kubernetes Engine (3 node pools, 12–30 nodes) with horizontal pod autoscaling, canary and progressive rollouts, health probes, and automated rollback.”

Who this is for: anyone who needs to understand or explain this system without a Kubernetes background. Every term is defined the first time it appears. Nothing assumes prior platform-engineering knowledge.

Contents

1. **The project in plain English** — what was built, why, and what 99.95% really costs
2. **The architecture diagram, explained piece by piece** — every box and every arrow
3. **Every component, end to end** — the thirteen building blocks and what each one solves
4. **Every workflow, end to end** — a request, a release, a traffic spike, and a failure at 3 a.m.
5. **How the 99.95% is actually earned** — the arithmetic behind the number
6. **Glossary** — every term used, in one sentence each
7. **Explaining this out loud** — the 30-second, 2-minute, and deep-dive versions

Diagrams

- Diagram 1** — Overall platform architecture (landscape, follows Section 2)
- Diagram 2** — Pod lifecycle and the three health probes (landscape, follows Section 3.6)
- Diagram 3** — Release workflow, canary rollout with automated rollback (follows Section 4.2)
- Diagram 4** — Traffic-spike workflow, five stages of autoscaling (landscape, follows Section 4.3)

1. The Project in Plain English

1.1 The whole thing in one paragraph

A company's product is not one big program. It is built from more than 25 small, independent programs called **microservices** — one handles logins, one handles payments, one handles search, and so on. Each of those was packaged into a **container** (a sealed box holding the app plus everything it needs to run) and handed to **Kubernetes**, the system that decides which machine each container runs on and restarts it when it dies. The specific Kubernetes offering used here is **Oracle Kubernetes Engine**, or OKE, where Oracle runs the brain of the cluster and we run the machines that do the work.

The job described by the bullet point was **not** writing those 25 applications. It was building the layer underneath them — the platform — so that they stay available. And “stay available” was given a hard number: **99.95%**. Systems almost never break at random; they break at two predictable moments — when new code is shipped, and when traffic suddenly surges. So the platform was designed around those two moments specifically. New code goes out to a tiny slice of users first and undoes itself automatically if it misbehaves. Capacity grows on its own when traffic climbs, and shrinks again when it falls. Nobody has to be awake for either.

1.2 What “99.95% availability” actually costs you

Availability sounds abstract until you convert it into minutes. The percentage is the share of time the service answers requests correctly. The remainder is your **downtime budget** — the total outage you are permitted before you have broken your promise.

Availability	Downtime per day	Downtime per month	Downtime per year
99% (“two nines”)	14 min 24 s	7 h 18 min	3 d 15 h
99.9% (“three nines”)	1 min 26 s	43 min 50 s	8 h 46 min
99.95% — this project	43 s	21 min 54 s	4 h 23 min
99.99% (“four nines”)	8.6 s	4 min 23 s	52 min 36 s

Why that number changes how you build

Twenty-two minutes a month is the entire budget — for everything. Every deploy, every crashed process, every cloud hiccup, every database failover comes out of the same 22 minutes.

One bad release that takes 25 minutes to notice and roll back by hand does not dent the budget. It **destroys the whole month's budget in a single incident**.

That is the reason this platform is built around automation rather than alerting. A human being paged at 3 a.m. needs a few minutes just to wake up, find a laptop and read the alert — and those minutes alone are a meaningful fraction of the monthly allowance. The system has to catch and reverse its own mistakes in under a minute, without waiting for anyone.

1.3 The three things that threaten uptime, and the answer to each

Everything in this platform exists to defeat one of exactly three threats. If you remember nothing else from this document, remember this table — every component later on maps back to one of these rows.

The threat	What goes wrong without a platform	How this platform answers it
Releases Shipping new code	New version is pushed to all users at once. If it is broken, 100% of customers see the breakage, and someone has to notice, decide, and manually revert.	Canary + progressive rollout. New code sees 5% of traffic first, then 25%, then 50%. Metrics are checked at each step. Automated rollback reverses it in under a minute if anything looks wrong.
Traffic spikes Sudden surge in load	A fixed number of servers gets overwhelmed. Requests queue, then time out, then fail. The site is technically “up” but useless.	Horizontal pod autoscaling adds more copies of a service within seconds. The cluster autoscaler adds more machines (12 → 30) within minutes. Spare capacity covers the gap between the two.
Random failure A machine or process dies	Whatever was running on that machine is simply gone. Users hitting it get errors until someone rebuilds it.	Every service runs at least 3 copies spread across 3 fault domains . Health probes detect a sick copy and pull it out of rotation or restart it. Kubernetes replaces what died automatically.

1.4 What “durable platform primitives” means

This is the phrase in the bullet point that sounds most like jargon, so it is worth unpacking properly. It has two halves.

- **Primitives** are reusable building blocks. With 25+ services, the wrong approach is to solve availability separately for each one — that is 25 different rollout scripts, 25 different guesses at probe settings, and 25 different ways to fail. The right approach is to build the pattern **once** as a standard template that every service inherits: a shared deployment chart, a standard set of health checks, a standard autoscaling rule, a standard rollout strategy. A new service gets all of it on day one for free.
- **Durable** means they keep working without supervision. A primitive that requires a person to watch a dashboard and press a button is not durable — it is a manual process wearing a costume. Durable means the rollback fires at 3 a.m. on a public holiday with nobody logged in, and the on-call engineer reads about it in the morning.

The practical result: an engineer on any of the 25 teams ships code by merging a pull request. They do not write rollout logic, do not configure autoscaling, and do not decide how to roll back. The platform supplies all of that, identically, to every service.

1.5 A note on the two numbers in the bullet

“3 node pools” — a **node** is one virtual machine. A **node pool** is a named group of identical machines managed together. Three pools means the machines were deliberately split into three groups with different jobs, so that different kinds of work cannot interfere with each other. Section 3.3 covers what each group does and why the separation matters.

“12–30 nodes” — the cluster never drops below 12 machines and never grows past 30. The floor guarantees there is always enough capacity to survive a failure without scrambling; the ceiling is a cost guardrail that stops a runaway loop from ordering hundreds of machines. Section 3.4 explains how the number moves between them.

2. The Architecture Diagram, Explained

The full architecture is on the next page, printed landscape so nothing is cramped. Read it twice, in two different directions:

- **Top to bottom** follows a user's request: from the person's browser, through the load balancer, through the ingress router, into a pod, and down to the database. This is the **data path** — it is what carries live traffic.
- **Right to left** follows the automation: the pipeline that ships code, the controller that manages rollouts, the system that collects metrics, the autoscalers, and the rollback trigger. This is the **control path** — it carries no user traffic at all. Its only job is watching the data path and correcting it.

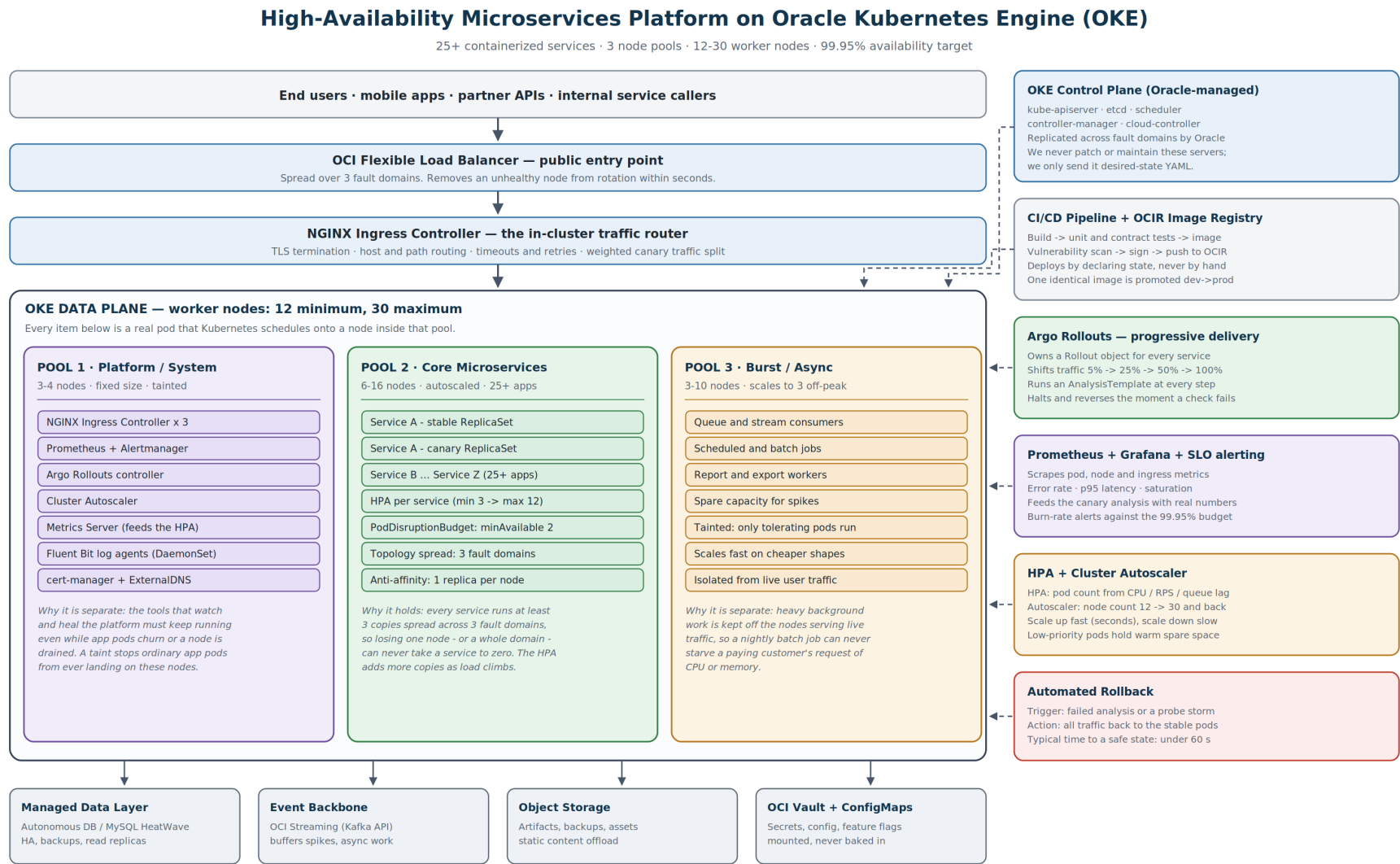
That split is the single most important idea in the diagram. **If the entire control path vanished right now, users would notice nothing** — traffic would keep flowing. What would stop is the system's ability to heal itself, deploy safely and grow. The control path is not in the way of your request; it is standing beside it, watching.

Reading tip

Dashed arrows mean “observes or controls.” Solid arrows mean “traffic flows this way.” The right-hand column connects to the cluster only with dashed lines — that is the control path, and it is deliberately drawn as separate from the request path.

Diagram 1 — Overall Platform Architecture

Read top-to-bottom for the path a request takes. Read right-to-left for the automation that keeps that path healthy.



Read the diagram top-to-bottom for the path a request takes; read right-to-left for the automation that keeps that path healthy.

Solid arrows carry user traffic. Dashed arrows observe or control. Section 2.1 explains every box in this diagram.

2.1 Layer by layer, every box in the diagram

Top box — the callers

Everything that talks to the platform: end users in browsers, mobile apps, partner companies calling the public API, and the services' own internal traffic to each other. It is drawn as one box because from the platform's point of view they are all just requests arriving. It matters that internal callers are included — in a 25-service system, most traffic is services calling other services, not humans.

Second box — OCI Flexible Load Balancer

The **public front door**. It owns the public IP address that DNS points at, and it spreads incoming requests across the worker machines. Two properties make it an availability component rather than just a router:

- It runs **across three fault domains** — separate racks with independent power and network inside an Oracle data centre. Losing one rack does not lose the load balancer.
- It **health-checks its own backends**. It continuously pings each worker node, and a node that stops answering is pulled out of rotation within seconds. This is the outermost of several safety nets: even if everything inside Kubernetes failed to notice a dead machine, the load balancer would already have stopped sending it traffic.

“Flexible” is Oracle's name for the shape whose bandwidth range you set yourself, so it scales with the traffic spikes rather than being a fixed-size bottleneck in front of an elastic cluster.

Third box — NGINX Ingress Controller

The load balancer knows how to reach the **cluster**. The ingress controller knows how to reach the **right service inside it**. It is itself a set of pods running in the cluster, and it does four jobs:

- **TLS termination** — decrypts HTTPS once at the edge so the 25 services do not each have to manage certificates.
- **Host and path routing** — reads the URL and forwards `/api/payments` to the payments service and `/api/search` to the search service.
- **Timeouts and retries** — gives up on a slow backend rather than holding the connection open forever, and retries a request that failed on a pod that died mid-flight. This turns many would-be user-visible errors into a slightly slower but successful response.
- **Weighted traffic splitting** — the capability the entire canary strategy depends on. It can be told “send 5% of requests for this service to this specific group of pods, and 95% to that one.” Without this, progressive rollout is not possible.

The large outlined area — the OKE data plane

This is the pool of worker machines that actually run application containers: **12 at minimum, 30 at maximum**.

Everything drawn inside it is a real **pod** — the smallest unit Kubernetes schedules, usually one running container. The three coloured columns inside are the three node pools.

Pool 1 (purple) — Platform / System

Three to four machines, a **fixed** size, running the platform's own tooling rather than customer-facing applications: the ingress controllers, the metrics stack, the rollout controller, the autoscaler, the log collectors, and the certificate manager. These machines carry a Kubernetes **taint**, which is a marker meaning “ordinary pods are not allowed here unless they explicitly ask to be.”

Why isolate them: these are the components that observe and heal everything else. If they compete for CPU with a busy application, they slow down exactly when they are needed most — during an incident. Worse, if a traffic spike caused the autoscaler or the metrics stack to be evicted to make room, the system would lose its ability to respond to the very spike that displaced it. Keeping the pool small and fixed also means it is predictable: it never scales, so it never surprises you.

Pool 2 (green) — Core Microservices

Six to sixteen machines, autoscaled, running the 25+ user-facing services. Notice what is drawn in this column: not just the applications, but the **rules that constrain how they are placed**.

- **Stable and canary ReplicaSets side by side** — during a release, two versions of the same service run simultaneously, and the ingress decides how much traffic each receives. A **ReplicaSet** is simply Kubernetes' record of “keep N identical copies of this exact version running.”
- **An HPA per service** — minimum 3 copies, maximum 12, adjusted automatically based on load.
- **PodDisruptionBudget: minAvailable 2** — a rule that blocks any voluntary operation which would drop a service below 2 running copies.
- **Topology spread across 3 fault domains** — copies must be distributed across racks, not bunched into one.
- **Anti-affinity: 1 replica per node** — two copies of the same service may not share a machine, so one machine dying can never remove two copies at once.

Those last three are the reason a node failure is a non-event here. Three copies on three racks with a floor of two available means the platform can lose an entire rack and still be serving.

Pool 3 (amber) — Burst / Async

Three to ten machines on cheaper shapes, running work that is not a user waiting for a response: queue consumers, scheduled jobs, report generation, exports. Also tainted, so nothing lands here by accident.

Why isolate them: background work is greedy and bursty. A nightly report that reads a million rows will happily consume every spare CPU cycle on its machine. If that machine were also serving login requests, users would feel it. Separating the pools means the worst a runaway batch job can do is slow down other batch jobs. This pool also doubles as **shock absorber** — it holds spare capacity that can be released quickly when a spike hits.

The right-hand column — the control path

Six boxes, dashed-connected, none of which touch user traffic:

- **OKE Control Plane (Oracle-managed)** — the cluster's brain: the API server that receives instructions, the etcd database that stores the desired state, the scheduler that assigns pods to nodes, and the controllers that continuously drive reality toward the desired state. Oracle runs, patches and replicates this. That is the practical value of a managed Kubernetes service: the hardest part to operate is somebody else's responsibility.
- **CI/CD pipeline + OCIR registry** — builds each service into a container image, tests it, scans it for known vulnerabilities, signs it, and stores it in Oracle's container registry tagged with the git commit hash. The same image is promoted through every environment, so what was tested is bit-for-bit what runs in production.
- **Argo Rollouts** — the progressive delivery controller. It owns the rollout of every service and is the component that shifts traffic in steps and aborts on failure.
- **Prometheus + Grafana + SLO alerting** — collects metrics from every pod, node and the ingress. It serves two masters: humans looking at dashboards, and the rollout controller asking “is this canary healthy?”

- **HPA + Cluster Autoscaler** — the two independent scaling brains. One changes the number of pods; the other changes the number of machines.
- **Automated rollback** — not really a separate product but the *behaviour* that emerges when the rollout controller trusts the metrics stack enough to act on it without asking.

The bottom row — where state lives

Deliberately drawn **outside** the cluster. Pods are disposable: they are created, destroyed and moved constantly. Anything that must survive that churn lives in a managed service instead.

- **Managed database** (Autonomous Database or MySQL HeatWave) — highly available with its own backups, failover and read replicas.
- **OCI Streaming** — a Kafka-compatible message queue. Its role in availability is important: during a spike, work that does not need an immediate answer is written to the queue and processed later. This lets the system **absorb** a surge instead of buckling under it.
- **Object Storage** — build artefacts, backups, and static assets served without touching a pod.
- **OCI Vault + ConfigMaps** — secrets and configuration, mounted into pods at runtime and never baked into the image. That is why one identical image is safe to promote from test to production.

2.2 The single sentence that summarises the whole diagram

User traffic flows down the left. Automation watches from the right. State lives at the bottom, outside the churn. Every design decision in this platform is an application of that separation — keep the request path simple and fast, keep the machinery that heals it out of its way, and keep anything you cannot afford to lose somewhere that does not get restarted.

3. Every Component, End to End

Thirteen building blocks. For each one: **what it is, what it actually does, what breaks without it, and how it was configured here**. They are ordered roughly from the outside in — from the things you can point at, to the invisible rules that hold it all together.

3.1 The 25+ containerized microservices

What it is

A **microservice** is one small application that does one job and talks to the others over the network. Instead of a single program handling logins, payments, search, notifications and reporting, you get five programs — each owned by a team, released on its own schedule, and scaled independently.

A **container** is that application packaged with its runtime, libraries and dependencies into one sealed, read-only image. The point of a container is that it behaves identically on a laptop, in a test cluster and in production, because it carries its own environment with it. “It worked on my machine” stops being a sentence anyone says.

What it does for availability

Splitting into services converts one enormous shared risk into 25 small independent ones. If the reporting service crashes in a monolith, the whole product goes down with it. Here, reporting goes down and logins keep working. This is called limiting the **blast radius** — the amount of damage one failure is allowed to cause — and it is the foundational availability decision that everything else builds on.

The trade-off, stated honestly

Microservices do not make a system more reliable for free. They **multiply the number of things that can fail** and replace fast, reliable in-process function calls with slow, fallible network calls. Twenty-five services means 25 deployments to get right, 25 sets of health checks, and a web of dependencies where a slow service can stall its callers. The architecture is only a net win **if** the platform underneath it is strong enough to manage the multiplication — which is precisely why the platform work described in this document exists at all.

3.2 Oracle Kubernetes Engine (OKE)

What Kubernetes is, without the jargon

Kubernetes is a system you give a **desired state** to, in a file: “I want 5 copies of the payments service, version 2.3, each with 1 CPU and 2 GB of memory, reachable at this address.” Kubernetes then works continuously to make reality match that description. If a copy dies, it starts another. If a machine disappears, it moves that machine's work elsewhere. Nobody tells it **how** — they only tell it **what**.

This is called **declarative** management, and it is the reason self-healing works. You are not running a script that does a deployment once; you are stating a fact that a controller enforces forever.

Control plane vs data plane

Every Kubernetes cluster has two halves, and knowing which is which explains what a “managed” service actually buys you.

	Control plane — the brain	Data plane — the muscle
What it holds	API server, the etcd database of desired state, the scheduler, the controller managers.	The worker nodes: ordinary virtual machines that run the actual application containers.
What it does	Accepts instructions, records what <i>*should*</i> be true, decides which node each pod goes on, and continuously corrects drift.	Runs the containers, serves user traffic, reports its health back to the control plane.
Who operates it	Oracle. Replicated across fault domains, patched and upgraded by them. Not visible to us as servers.	Us. We choose the machine sizes, how many, and how they are grouped into pools.
If it goes down	Running pods keep serving traffic. What stops is <i>*changes*</i> — no new deployments, no rescheduling, no autoscaling.	Whatever was on the failed node stops. The control plane notices and recreates it elsewhere.

That third row is the whole argument for OKE. Running a highly available etcd cluster correctly is one of the harder jobs in infrastructure, and it produces no customer value when done well — only pain when done badly. Handing it to Oracle removes an entire category of failure from the team's plate and leaves them to focus on the data plane, where the design choices actually differentiate anything.

The last row is worth internalising too: **a control plane outage is not a customer outage**. The cluster becomes temporarily unable to change, but it does not stop working.

3.3 Three node pools — and why not one

What a node pool is

A **node** is one virtual machine that runs pods. A **node pool** is a named group of identical nodes — same shape, same configuration, same scaling rules — managed as a unit. Splitting one cluster into several pools lets you run different classes of work on differently-tuned hardware, with different scaling behaviour, and keep them from interfering.

Pool	Size	What runs there	Why it is separate
1. Platform / System	3–4 nodes fixed, tainted	Ingress controllers, Prometheus, Argo Rollouts, cluster autoscaler, log agents, cert-manager, metrics server.	These heal and observe everything else, so they must never be starved of CPU or evicted by a busy application — least of all during an incident, when they matter most.
2. Core Microservices	6–16 nodes autoscaled	The 25+ user-facing services, in stable and canary versions, governed by HPAs, disruption budgets and spread rules.	This is the revenue path. It gets the reliable machine shapes, the strictest placement rules, and the most conservative scale-down behaviour.
3. Burst / Async	3–10 nodes tainted	Queue and stream consumers, scheduled and batch jobs, report and export workers, plus spare surge capacity.	Background work is bursty and greedy. Isolating it means a runaway batch job can only ever slow down other batch jobs — never a customer's request.

What one pool would cost you

- **Noisy neighbours.** A nightly report job and a login request compete for the same CPU. Logins get slower at exactly midnight, every night, and it is hard to explain why.
- **Wrong hardware for everything.** Batch jobs want cheap, memory-heavy machines; latency-sensitive APIs want fast, consistent CPU. One pool forces one compromise that fits neither.
- **Scaling that fights itself.** Batch work wants to scale hard and fast; the platform stack wants to never move. In one pool, the autoscaler's decisions for one workload constantly disrupt the other.
- **No safe place to stand.** During a spike, the monitoring stack and the autoscaler could themselves be squeezed out by application pods — losing exactly the tools needed to handle the spike.

3.4 12–30 nodes and the Cluster Autoscaler

What it is

The **Cluster Autoscaler** is a controller that changes the number of **machines**. It does exactly two things, and understanding the trigger for each is the key to the whole component.

- **Scale up:** it watches for pods stuck in **Pending** state. A pod is Pending when Kubernetes has accepted it but cannot find any node with enough free CPU or memory to place it. To the autoscaler, a Pending pod is a direct, unambiguous statement that the cluster is too small — so it asks Oracle for another node in the appropriate pool.
- **Scale down:** it watches for nodes that have been under-used for a sustained period (typically ten minutes) and whose pods could all fit elsewhere. It then **cordons** the node (marks it as accepting no new pods), **drains** it (evicts the pods gracefully so they restart elsewhere first), and only then deletes it.

Why the floor of 12 matters

The minimum is not “the smallest number that works.” It is sized so that the cluster can lose a whole fault domain — roughly a third of its machines — and still run every service at its minimum replica count without needing to scale first. Scaling takes minutes; a rack failure takes zero notice. The floor is the capacity that is standing ready before anything goes wrong.

Why the ceiling of 30 matters

The maximum is a **blast radius control for cost, and a circuit breaker for bugs**. Autoscaling is a feedback loop, and a bug in an application — an infinite retry loop, a memory leak that inflates CPU — can look exactly like genuine demand. Without a ceiling, the cluster would obediently order machines until someone noticed the invoice. The cap converts a runaway cost incident into a capacity alert.

The important limitation

Nodes are slow. Provisioning a new virtual machine, booting it, joining it to the cluster, and pulling container images takes **one to three minutes**. A traffic spike arrives in seconds. This gap is the single most important operational fact about cluster autoscaling, and it is why the platform does not rely on it alone — it is the slow, deep reserve behind the fast responder described next.

3.5 Horizontal Pod Autoscaler (HPA)

What it is

The HPA changes the number of **copies of a service** — pods — rather than the number of machines. “Horizontal” means adding more copies, as opposed to “vertical” scaling, which means making one copy bigger. Horizontal is preferred for availability for a simple reason: three small copies survive a machine failure, while one large copy does not.

How it actually works, step by step

- Every 15 seconds the HPA reads a metric for the service's pods — average CPU usage, requests per second, or queue lag.
- It compares that against a target. Here the target was roughly **65% CPU**, chosen deliberately below 100% so there is headroom to absorb a surge while new pods are still starting.
- It calculates the desired count with a simple ratio: $\text{desired} = \text{current} \times (\text{actual} \div \text{target})$. Three pods averaging 90% against a 65% target gives $3 \times (90 \div 65) \approx 4.2$, rounded up to 5.
- It writes that number to the ReplicaSet, and Kubernetes creates the new pods. Each must pass its readiness probe before receiving any traffic.

Scale up fast, scale down slow

The two directions are deliberately asymmetric, and this is one of the most important configuration decisions in the platform.

Direction	Behaviour	The reasoning
Scaling up	Immediate. Reacts within roughly 15–30 seconds of the metric crossing the target, and may double the pod count in one step.	Being slow to add capacity means dropped requests. Adding capacity you turn out not to need costs a few cents. The costs are not symmetric, so the response should not be either.
Scaling down	Waits for a 5-minute stabilisation window in which the metric stays low, then removes pods gradually.	Traffic is spiky by nature. Removing pods the instant load dips causes thrashing — scaling down, immediately needing them again, and scaling back up — which produces constant churn and a stream of cold-start latency spikes for users.

How the HPA and the Cluster Autoscaler cooperate

They never talk to each other. They are two independent loops that happen to compose correctly, and the chain is worth stating explicitly because it is a common point of confusion:

Traffic rises → pod CPU rises → **HPA** adds pods → pods fit in spare room on existing nodes → done in **seconds**.
Traffic rises further → **HPA** adds more pods → no node has room → pods go **Pending** → **Cluster Autoscaler** sees Pending pods and adds nodes → pods get placed → done in **minutes**.

The Pending state is the entire interface between them. The HPA does not know machines exist. The autoscaler does not know traffic exists. One creates demand it cannot satisfy; the other notices unsatisfied demand and buys hardware.

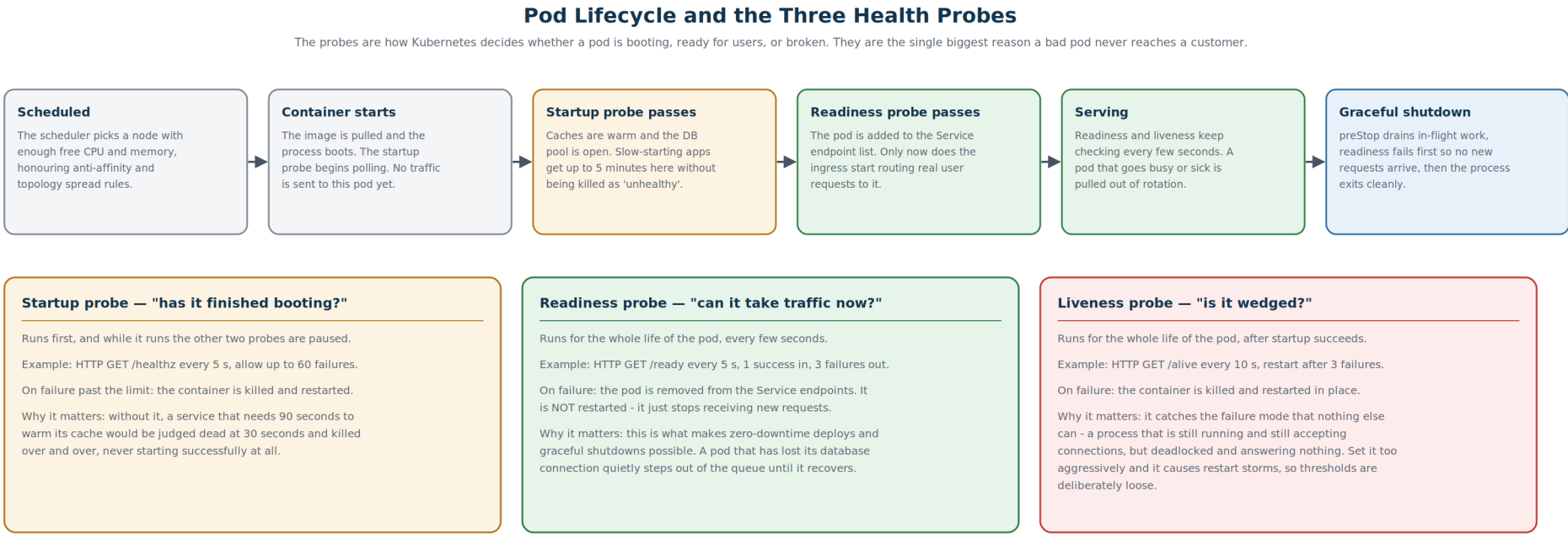
The gap between “seconds” and “minutes” is covered by two things: the 65% CPU target, which leaves headroom in the pods that already exist, and low-priority placeholder pods that reserve empty space and can be evicted instantly to make room for real work.

3.6 Health probes

Health probes are how Kubernetes finds out whether a pod is booting, ready for users, or broken. They are the cheapest and highest-leverage availability feature in the entire system — and the most commonly misconfigured. The three probes, the pod lifecycle they govern, and their failure behaviour are laid out in the diagram on the following page.

Diagram 2 — Pod Lifecycle and the Three Health Probes

The top row is the life of a single pod. The three cards below explain what each probe checks and what happens when it fails.



The critical distinction most people get wrong

Readiness controls traffic. Liveness controls restarts. They sound similar and behave completely differently, and confusing them causes outages.

A pod that fails its **readiness** probe is quietly removed from the list of pods receiving traffic. It keeps running. Nothing is killed. When it recovers, it is added back. This is the correct response to a temporary problem — a lost database connection, a full request queue, a dependency that is slow right now.

A pod that fails its **liveness** probe is **killed and restarted**. This is the correct response to a permanently stuck process — a deadlock, an exhausted thread pool, a wedged event loop.

The classic self-inflicted outage

A team points the liveness probe at an endpoint that also checks the database. The database has a brief hiccup. **Every pod of every service** fails its liveness probe simultaneously, so Kubernetes restarts all of them at once. Now the database, already struggling, is hit by a stampede of cold pods all reopening connection pools at the same moment — and the brief hiccup has become a full outage that the platform caused itself.

The rule that prevents it: **a liveness probe must only ever test the process itself**, never its dependencies. Dependency health belongs in the readiness probe, where the response is to stop taking traffic rather than to commit suicide.

Why the startup probe exists

Some applications take a long time to become useful — a JVM service warming caches and opening connection pools might need 90 seconds. Without a startup probe, the liveness probe begins immediately, judges the still-booting app to be dead at 30 seconds, and kills it. It restarts, gets killed again, and enters an endless **CrashLoopBackOff** — never once having got far enough to work. The startup probe simply pauses the other two probes until the application says it has finished booting, allowing a generous window once, at the start, without loosening the checks that run for the rest of the pod's life.

How probes create zero-downtime deployments

This is the mechanism that makes a release invisible to users, and it deserves to be spelled out because it is not obvious:

- A new pod starts. Because its readiness probe has not yet passed, **it receives no traffic at all** — so a slow or broken new version cannot serve a single user while it is starting.
- Only when the app reports ready is the pod added to the service's endpoint list and given traffic.
- When an old pod is asked to shut down, the **readiness probe is failed first**. The pod is removed from the endpoint list, so no new requests arrive, while requests already in flight are allowed to finish.
- Only after that drain does the process actually exit.

The result is that at every moment during a deploy, every pod receiving traffic is a pod that has declared itself ready. There is no window where a request can be routed to something that is not prepared to answer it.

3.7 Canary and progressive rollouts

What it is

The name comes from the canaries once carried into coal mines: a small, sensitive thing sent ahead to detect danger before it reaches everyone else. A **canary release** sends the new version of a service a tiny slice of real production traffic while the old version continues serving the rest. A **progressive rollout** is the full sequence — increasing that slice in stages, pausing to measure at each one.

Why not just deploy it?

Because testing before release only catches what you thought to test for. Production has real user behaviour, real data volumes, real concurrency, real dependency latency and real edge cases that no test environment reproduces faithfully. Some bugs only exist under production conditions, and the only honest way to find them is to expose production traffic to the new code — **but in a controlled dose, with a measurement, and with a way back.**

The actual sequence used here

Step	Traffic to new version	What happens	Users at risk
Deploy canary	0%	One new pod starts. Startup and readiness probes must pass before it is eligible for anything.	None
Step 1	5%	Analysis runs for ~5 minutes. Error rate and p95 latency are compared against the stable version's live baseline.	1 in 20
Step 2	25%	Same checks, longer sample. Enough volume now to reveal problems that only appear under load.	1 in 4
Step 3	50%	Final confirmation at real scale, including any resource pressure the new version creates.	1 in 2
Promote	100%	Canary becomes the new stable. The old ReplicaSet is kept briefly, scaled to zero, so a revert stays instant.	All — but now with strong evidence

What “analysis” actually means

This is the part that turns a canary from theatre into a real safety mechanism. At each step, Argo Rollouts queries Prometheus with concrete, pre-agreed queries and compares the results against thresholds. Typical checks:

- **Error rate:** the share of the canary's responses that are HTTP 5xx must stay below 0.5%.
- **Latency:** the canary's p95 response time must not be more than 20% worse than the stable version's. Comparing against the **live** stable version, rather than a fixed number, is important — it means a general slowdown affecting both versions does not falsely blame the new code.
- **Stability:** no pod restarts, no liveness probe failures, no rising memory that suggests a leak.

If every check passes, the rollout proceeds to the next step on its own. If any check fails, it aborts — covered next.

Why this is more effective than blue-green

Blue-green deployment runs two complete environments and flips 100% of traffic from one to the other at once. It gives a fast revert, but it does not reduce exposure: at the moment of the switch, either everybody is on the old version or everybody is on the new one. If the new version is bad, 100% of users hit it until someone flips back. Progressive rollout limits the number of people who ever see a bad version to the size of the current step — which is the difference between an incident and a footnote.

3.8 Automated rollback

What it is

The rollout controller does not merely *report* a failed analysis — it acts on it. When a check fails, it immediately sets the canary's traffic weight to 0%, scales the canary ReplicaSet to zero, and pages on-call to explain what it already did.

The alert is a notification, not a request for permission.

Why automation is not optional at 99.95%

Work through the timings of the manual alternative. An alert fires. The pager takes 1–2 minutes to reach a sleeping engineer. They wake up, find a laptop, connect to the VPN and read the alert: 3–5 minutes. They open dashboards and confirm it is real rather than a false alarm: 3–5 minutes. They decide, find the right command and run it: 2–3 minutes. The rollback itself takes 1–2 minutes.

Manual response: 10–17 minutes of degraded service — half to three-quarters of the entire monthly downtime budget, consumed by one incident, on a good night with a responsive engineer.

Automated response: under 60 seconds, affecting only the small percentage of traffic that was on the canary. The month's budget is essentially untouched.

This is the specific mechanism by which 99.95% is achieved during releases. It is not that fewer bugs are written — it is that bugs are contained to a slice of traffic and reversed faster than a human could even read the alert.

What makes rollback safe and instant here

- **The stable pods were never touched.** A canary rollout adds new pods alongside the old ones; it does not modify or replace them. “Rolling back” is therefore just a routing change — stop sending traffic to the new pods. There is nothing to rebuild, restore or reinstall.
- **Database migrations are backwards-compatible by rule.** This is the discipline that makes the rest possible. Schema changes are always additive — add a column, never rename or drop one in the same release — so the old version of the code still runs correctly against the new schema. Without this rule, rolling back the code while the database has already moved forward would break things worse than the bug did.
- **Configuration is versioned with the code**, so reverting the deployment reverts its settings too.

3.9 Placement rules: replicas, anti-affinity and topology spread

Three copies of a service are worthless if all three are on the same machine. These rules control *where* copies are placed, and they are what convert a replica count into genuine fault tolerance.

- **Minimum 3 replicas.** Two is the intuitive minimum but is not enough: during a rolling update one is briefly unavailable, leaving a single point of failure. Three means the service survives a failure *while* being updated.

- **Pod anti-affinity.** A rule stating that two pods of the same service may not be scheduled onto the same node. Without it, the scheduler will happily pack all three copies onto whichever machine has the most free space — producing three copies with one shared fate.
- **Topology spread constraints.** A stronger version of the same idea applied to **fault domains** — groups of hardware with independent power and networking. The rule requires copies to be distributed evenly across all three, so the loss of an entire domain removes only one copy of any service.

Together these guarantee that the loss of one node removes at most one copy of any service, and the loss of one fault domain removes at most one copy of any service. Neither event can take a service to zero.

3.10 PodDisruptionBudget

A **PodDisruptionBudget** (PDB) is a rule that limits how many copies of a service may be voluntarily taken down at the same time. Here it is set to `minAvailable: 2` — at least two copies must remain available at all times.

The word **voluntary** carries the meaning. A PDB cannot stop a machine from catching fire; nothing can. What it stops is the platform harming itself through its own routine maintenance. When the cluster autoscaler drains a node to remove it, or a node is drained for an OS upgrade, Kubernetes checks every PDB first and refuses any eviction that would breach one. The drain waits instead.

The failure this prevents

Off-peak traffic drops, and the autoscaler decides to remove four under-used nodes. Without PDBs it drains them concurrently, and a service whose three copies happened to sit on three of those nodes goes to zero replicas for the 20–30 seconds it takes them to restart elsewhere.

A quiet Sunday night, no incident, no bad code — and a real outage caused purely by cost optimisation. With PDBs, the drains simply queue up and proceed one at a time, and users never notice.

3.11 Resource requests and limits

Every pod declares two numbers for CPU and memory. They do different jobs and both matter:

- **Request** — the amount reserved for this pod. The scheduler uses it to decide which node has room, and the node guarantees it. This is the number that makes placement predictable.
- **Limit** — the ceiling. Exceed the CPU limit and the pod is throttled; exceed the memory limit and it is killed (**OOMKilled**). This is the number that stops one leaking pod from taking down the machine and everything else on it.

Getting these right is unglamorous and disproportionately important. Requests set too low let the scheduler overpack a node until everything on it slows down together. Requests set too high waste money and force the cluster autoscaler to buy machines that are mostly idle. They are also what makes the HPA meaningful — “65% CPU utilisation” is measured **against the request**, so a wrong request makes every autoscaling decision wrong in the same direction.

3.12 Graceful shutdown

When a pod must stop — during a deploy, a scale-down or a node drain — shutting it down abruptly would kill requests that are mid-flight, and every one of those is a user-visible error. The platform enforces an ordered sequence instead:

- Kubernetes removes the pod from the service's endpoint list and runs a **preStop** hook, typically a short sleep. This pause matters more than it looks: endpoint changes take a few seconds to propagate to every ingress and proxy, and the sleep covers that window so no traffic is sent to a pod that is already leaving.
- The container receives **SIGTERM**, the polite request to stop. The application stops accepting new work but finishes what it is already handling.
- Kubernetes waits out the **termination grace period** (30 seconds here) for the process to exit on its own.
- Only if it has not exited by then is it forcibly killed with SIGKILL.

Without this, a routine scale-down after a traffic spike would generate a burst of errors every single time — the platform's own housekeeping showing up as failures in the availability numbers.

3.13 Observability, SLOs and the error budget

You cannot sustain a number you do not measure, and “99.95%” is a claim that requires evidence. The observability stack collects metrics from every pod, node and ingress: request rates, error rates, latency percentiles, CPU and memory, restart counts, and queue depths.

Those feed an **SLO** — a Service Level Objective, the formal target of 99.95% — and from it the **error budget**: the 0.05% of requests that are permitted to fail, roughly 22 minutes a month. Framing it as a budget rather than a prohibition changes how teams behave. A budget is something you are allowed to **spend**, deliberately, on shipping features. It converts the endless argument between “move fast” and “be careful” into an arithmetic question: how much of this month's budget is left?

The alerting is built on **burn rate** rather than raw thresholds — how fast the budget is being consumed. A slow trickle of errors is a ticket for business hours. A rate that would exhaust the month's budget within an hour is a page, immediately. This is what stops alert fatigue: engineers are woken for things that genuinely threaten the target, and not for anything else.

Critically, this same metrics pipeline is what the canary analysis queries. **The system's own monitoring is what decides whether a release lives or dies** — which means the monitoring must be trustworthy, and it is continuously proven so by being used for a consequential decision on every single deploy.

4. Every Workflow, End to End

Components are nouns. Workflows are the verbs — what actually happens, in order, when something occurs. Four workflows cover essentially everything this platform does: serving a request, shipping a release, absorbing a spike, and surviving a failure.

4.1 Workflow A — one request, from browser to database and back

This is the path that must never break. Roughly 40–80 milliseconds, end to end.

Step 1 — DNS resolution

The user's browser looks up `api.example.com` and receives the public IP address of the OCI Load Balancer. Nothing about the cluster is exposed here; the internals can be rebuilt entirely without this answer changing.

Step 2 — The load balancer chooses a node

The request arrives at the load balancer, which picks a healthy worker node from its backend set. “Healthy” is not assumed — it is the result of continuous health checks, and a node that stopped answering was removed from the set seconds ago. The choice is spread across fault domains, so consecutive requests may land in different racks.

Step 3 — The ingress controller routes it

Inside the cluster, the ingress controller terminates TLS and reads the request. Based on hostname and path it identifies the target service — say, payments. If a canary rollout is in progress for that service, **this is the moment the weighted split is applied**: a weighted random draw decides whether this particular request goes to the stable pods or the canary pods.

Step 4 — The Service picks a pod

A Kubernetes **Service** is a stable internal address in front of a changing set of pods. It holds an **endpoint list** — and, crucially, that list contains only pods currently passing their readiness probe. One pod is chosen from it. This is the exact point where all the health-probe machinery pays off: a pod that is starting up, shutting down, or temporarily unhealthy is simply not in the list, so it is not a candidate.

Step 5 — The application does the work

The pod handles the request: validates input, calls the managed database, perhaps calls two or three other microservices over the same internal Service mechanism. If part of the work does not need an immediate answer — sending a receipt email, updating an analytics warehouse — it is written to OCI Streaming and processed later by a consumer in Pool 3. That deferral is an availability decision: it keeps the user's response fast and keeps slow downstream systems off the critical path.

Step 6 — The response returns

The answer travels back out through the ingress and the load balancer to the browser. Along the way the request has been counted, timed and labelled by version. Those measurements land in Prometheus, where — if a canary is running — **this very request becomes one data point in the decision about whether the new version is allowed to proceed**.

What made this survivable at every step

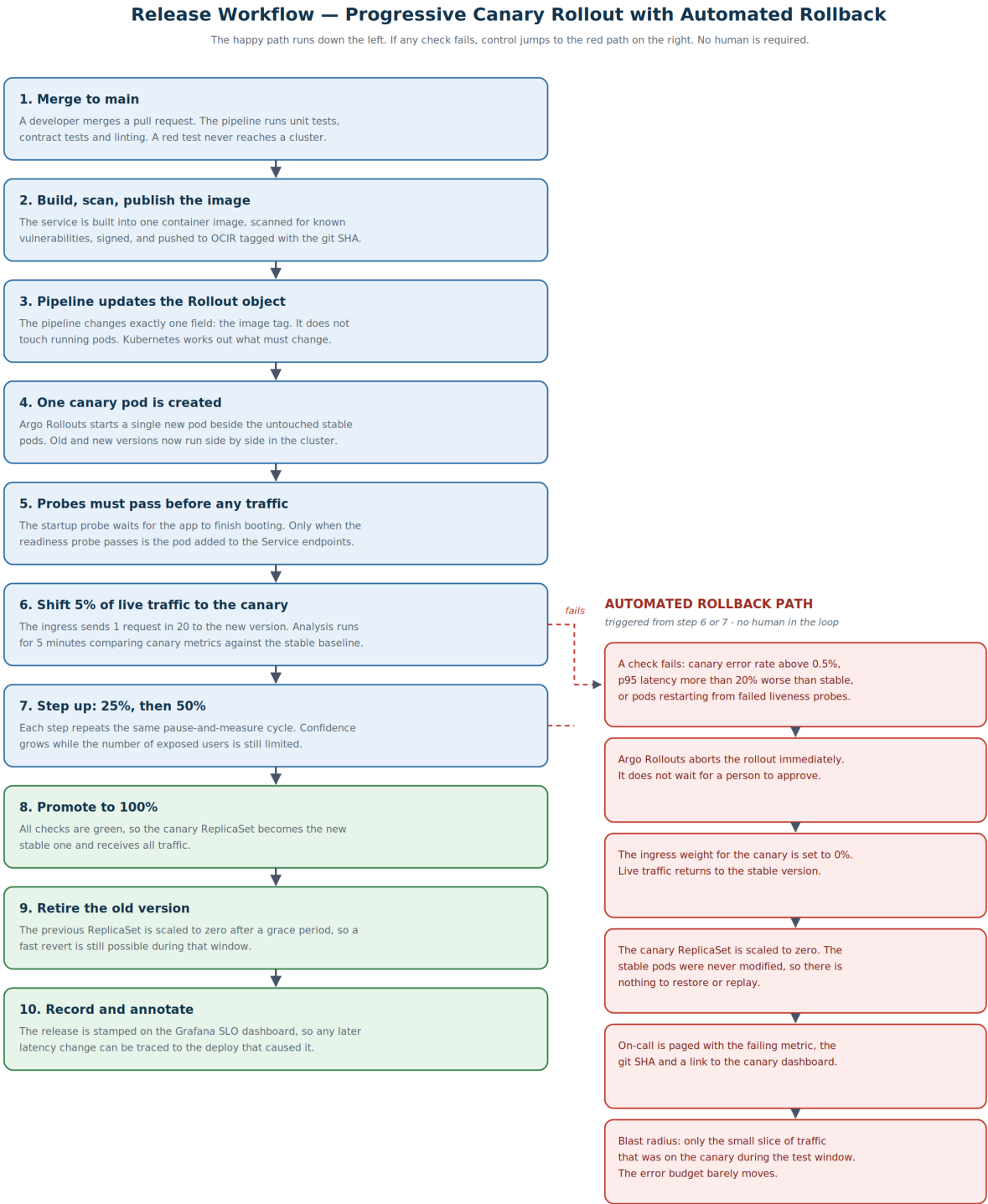
If this fails...	...the user still gets an answer, because
The chosen worker node is dead	The load balancer's health check already removed it from the backend set.
The chosen pod is unhealthy	It failed its readiness probe and was removed from the Service endpoint list.
The pod dies mid-request	The ingress retries the request against a different pod.
An entire fault domain is lost	Topology spread guarantees other copies exist in the two remaining domains.
A downstream dependency is slow	Non-essential work was already deferred to the queue rather than awaited inline.

4.2 Workflow B — shipping a release

The full sequence is on the next page: the normal path down the left, and the automated rollback path on the right. What follows here is the narration.

Diagram 3 — Release Workflow

The normal path runs down the left; the automated rollback path branches to the right.



The normal path, in words

Steps 1–3 — build and declare. A developer merges a pull request; nothing else about their job changes. The pipeline runs tests, builds one container image, scans it for known vulnerabilities, signs it and pushes it to the registry tagged with the git commit hash. Then it changes exactly one thing in the cluster: the image tag on the service's Rollout object. It does not touch a single running pod. It states a new desired reality and lets the controllers work out the difference — the declarative model from Section 3.2 doing its job.

Steps 4–5 — one pod, no traffic. Argo Rollouts creates a single canary pod beside the untouched stable pods. Two versions of the service are now running at once. The canary receives no traffic whatsoever until its startup probe confirms it has booted and its readiness probe confirms it can serve. A version broken badly enough to fail to start never reaches a user at all, and the rollout stalls harmlessly.

Steps 6–7 — measure, then commit further. The ingress is told to send 5% of live traffic to the canary. Analysis runs for about five minutes, comparing the canary's error rate and p95 latency against the stable version's live baseline. Green, and the weight steps to 25%, then 50%, each with the same pause-and-measure cycle. Confidence is bought incrementally, and the number of exposed users only grows as the evidence does.

Steps 8–10 — promote and record. All checks green, so the canary ReplicaSet becomes the new stable one and takes 100% of traffic. The previous ReplicaSet is scaled to zero but not deleted immediately, keeping an instant revert available during the grace period. Finally the release is annotated on the SLO dashboard, so any latency change discovered next week can be traced back to the deploy that introduced it.

The rollback path, in words

At any analysis step, one failing check is enough. The canary's error rate goes above 0.5%, or its p95 latency is more than 20% worse than stable, or its pods are restarting from failed liveness probes. The rollout aborts on the spot — no approval, no waiting.

The ingress weight for the canary drops to 0%, so live traffic returns to the stable version. The canary ReplicaSet is scaled to zero. **The stable pods were never modified at any point**, so there is nothing to restore, replay or repair — the system is already back in the state it was in before the deploy began. On-call is then paged with the failing metric, the git commit hash, and a link to the canary's dashboard, so the engineer starts their morning with a diagnosis rather than a mystery.

The number that matters

Total elapsed time from the first bad metric to full recovery: **under 60 seconds**. Total users affected: only the fraction that was on the canary during the test window — at the 5% step, one request in twenty, for a few minutes.

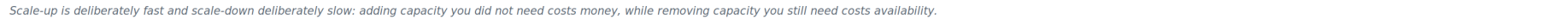
Compare that with a full deployment gone wrong: 100% of users affected for as long as it takes a human to notice, diagnose and revert.

4.3 Workflow C — absorbing a traffic spike

A marketing email goes out at 10 a.m. and traffic quadruples in seconds. The five stages of the platform's response are on the following page; the narration is below.

Five stages from steady state to surge and back, showing traffic, pod count and node count at each one.

Two independent autoscalers work at different speeds: pods appear in seconds, extra nodes arrive in minutes. Spare capacity covers the gap.



Stage by stage

Stage 1 — steady state. Every service runs its minimum of three pods, spread across three fault domains, on twelve nodes. CPU sits around 45%, comfortably below the 65% scale-up trigger. That gap is not waste; it is the reserve that covers the next twenty seconds.

Stage 2 — the spike hits (0–15 s). Requests jump fourfold almost instantly. No autoscaler has reacted yet — nothing could react that fast. The existing pods absorb the surge using the headroom between their resource request and their limit. Latency rises; errors do not. **This is why the CPU target was set at 65% rather than 90%:** the headroom is doing the work that no automation is fast enough to do.

Stage 3 — pods scale out (15–60 s). The HPA reads the new CPU and request-rate figures from the metrics server and roughly triples the replica count. The new pods fit into slack on the existing nodes, so no new machines are needed yet. Each new pod must pass its readiness probe before it is added to the Service endpoint list — so a pod that is still warming up never receives a request it cannot serve well.

Stage 4 — nodes scale out (1–3 min). Traffic keeps climbing and the HPA keeps adding pods until some cannot be placed and go **Pending**. The cluster autoscaler sees them and asks Oracle for more machines in pools 2 and 3, up to the ceiling of 30. In the interim, low-priority placeholder pods that were holding empty space are evicted instantly, giving real workloads somewhere to land while the new machines boot.

Stage 5 — scale back in (30 min+). The campaign burns out and traffic falls. Nothing happens immediately. Only after a five-minute stabilisation window does the HPA begin removing pods, gradually. The cluster autoscaler then waits for nodes to stay under-used for around ten minutes before cordoning, draining and deleting them — and every eviction during that drain is checked against the PodDisruptionBudgets, so no service is ever taken below two available copies by the cleanup.

The principle underneath all five stages

Scale up fast, scale down slow. Adding capacity you turn out not to need costs a small amount of money. Removing capacity you still need costs availability. The costs are wildly asymmetric, so the responses are deliberately asymmetric too.

4.4 Workflow D — a node dies at 3 a.m.

Nobody is awake. A worker node fails outright — hardware fault, hypervisor problem, kernel panic. It was running about eight pods belonging to six different services. Here is the full sequence, with nobody involved at any point.

Time	What happens	Effect on users
T+0 s	The node dies. Its eight pods stop instantly. Every affected service still has at least two other copies running in other fault domains, because anti-affinity and topology spread guaranteed it.	None. Traffic is already spread across the surviving copies.
T+2 to 10 s	The load balancer's health check fails against the dead node and removes it from the backend set. No further external traffic is sent there.	None.

Time	What happens	Effect on users
T+10 to 40 s	The Kubernetes control plane stops receiving heartbeats from the node's kubelet and marks it NotReady . Its pods are removed from every Service endpoint list, so internal callers stop routing to them.	Brief, if any — the ingress retries in-flight requests against healthy pods.
T+40 s to 2 min	Controllers notice the replica counts are below their declared minimums and schedule replacement pods on the remaining nodes. Each new pod boots and must pass its readiness probe before receiving traffic.	None. Capacity is being restored while the surviving copies serve normally.
T+2 to 5 min	If the remaining nodes lack room for the replacements, some pods go Pending, and the cluster autoscaler provisions a replacement node.	None.
Morning	An engineer reads an informational alert describing a node that failed and was replaced overnight. There is no incident to run and no backlog to clear.	None. The event never reached a customer.

This is what “durable” means in practice. Not that failures were prevented — a machine still died — but that the failure was **absorbed by design rather than resolved by a person**. The cost to the error budget was approximately zero.

5. How the 99.95% Is Actually Earned

The number is not a hope; it is the sum of specific decisions, each removing a specific quantity of expected downtime. This table is the argument in full.

Failure scenario	Without the platform	With the platform	Budget consumed
A bad release reaches production (the most common cause of self-inflicted outages)	100% of users hit the broken version. Detect, decide and manually revert: 10–20 minutes.	5–50% of traffic is exposed for a few minutes; automated rollback completes in under 60 s.	~0.5 min instead of ~15 min
A worker node fails	Whatever ran on it is gone until someone rebuilds or reschedules it.	Three replicas across three fault domains absorb it; replacements are scheduled automatically.	~0 min
Traffic quadruples suddenly	Fixed capacity is overwhelmed; requests queue, then time out. Degradation lasts as long as the spike.	Headroom absorbs the first 15 s, the HPA responds in under a minute, nodes follow within three.	~0 min
A process deadlocks	It keeps accepting connections and answering none. Users see hangs until someone restarts it.	The liveness probe fails three times and the container is restarted automatically.	~0 min for the service; one pod's share of traffic briefly
Routine node maintenance / scale-down	Concurrent drains can take a service to zero replicas without warning.	PodDisruptionBudgets serialise the drains; graceful shutdown drains in-flight requests first.	~0 min
A dependency turns slow	Callers block, thread pools fill, the failure cascades outward through the service graph.	Readiness probes take affected pods out of rotation; non-critical work was already deferred to the queue.	Degraded, not down

The one-sentence summary of the strategy

The platform does not try to prevent failure — it makes failure cheap. Every mechanism here either shrinks the number of users a failure can reach, or shrinks the time it takes to reverse, or removes the human being from the critical path. Three replicas across three fault domains shrink the reach. Canary steps shrink the reach. Automated rollback and self-healing shrink the time. Together they turn events that would each have cost 10–20 minutes into events that cost seconds — and 22 minutes a month becomes a budget you can comfortably live inside.

6. Glossary

Every term used in this document, in one sentence each.

Term	Plain-English meaning
Availability	The share of time a service answers requests correctly. 99.95% allows about 22 minutes of failure per month.
Blast radius	How much damage one failure is allowed to cause — how many users, and for how long.
Canary release	Sending a new version a small slice of real traffic first, to detect problems before everyone is exposed.
Cluster Autoscaler	The controller that adds and removes machines , triggered by pods that cannot be placed.
Container	An application packaged with everything it needs to run, so it behaves the same everywhere.
Control plane	The brain of a Kubernetes cluster — API server, state database, scheduler, controllers. Oracle runs this in OKE.
Cordon / drain	Marking a node so no new pods land on it, then moving its existing pods away gracefully before removing it.
CrashLoopBackOff	A pod stuck in a cycle of starting, failing and restarting, with Kubernetes waiting longer between each attempt.
Data plane	The worker machines that actually run application containers and serve traffic.
Declarative	Describing the state you want rather than the steps to reach it, and letting a controller continuously enforce it.
Error budget	The failure you are allowed before missing your target — 0.05% of requests, or about 22 minutes a month.
Fault domain	A group of hardware with independent power and networking, so a failure in one does not affect the others.
Graceful shutdown	Stopping a pod in order — stop new traffic, finish current requests, then exit — so no request is dropped.
HPA	Horizontal Pod Autoscaler: the controller that adds and removes copies of a service based on load.
Ingress controller	The in-cluster router that terminates TLS, matches URLs to services, retries failures, and splits canary traffic.
Liveness probe	A repeated check asking “is this process wedged?” Failing it restarts the container.
Microservice	One small application doing one job, communicating with others over the network.
Node / node pool	A node is one virtual machine; a node pool is a named group of identical nodes managed together.

Term	Plain-English meaning
OKE	Oracle Kubernetes Engine — managed Kubernetes where Oracle operates the control plane and you operate the nodes.
PodDisruptionBudget	A rule blocking any voluntary eviction that would drop a service below a stated number of copies.
Pending	A pod Kubernetes has accepted but cannot place because no node has room. The autoscaler's trigger to add machines.
Pod	The smallest unit Kubernetes runs and schedules — in practice, one running container.
Progressive rollout	Increasing a new version's traffic share in stages (5% → 25% → 50% → 100%), measuring at each step.
Readiness probe	A repeated check asking “can this pod take traffic right now?” Failing it removes it from routing without restarting it.
ReplicaSet	Kubernetes' record of “keep N identical copies of this exact version running.”
Request / limit	The resources a pod reserves, and the ceiling it may not exceed. Requests drive scheduling; limits contain damage.
Rollback	Returning to the previous working version. Here it is a routing change, not a redeploy, which is why it is fast.
Service (Kubernetes)	A stable internal address in front of a changing set of pods, listing only those currently passing readiness.
SLO	Service Level Objective — the formal reliability target, here 99.95%.
Startup probe	A one-time check that pauses the other probes until a slow-booting application has finished starting.
Taint / toleration	A marker on a node saying “only pods that explicitly accept this may run here,” used to reserve pools.
Topology spread	A rule forcing copies of a service to be distributed evenly across fault domains rather than bunched together.

7. Explaining This Out Loud

Three versions of the same story, at three depths. Use whichever fits the time you are given.

The 30-second version

“We ran more than 25 microservices on managed Kubernetes in Oracle Cloud, with a 99.95% availability target — about 22 minutes of downtime a month for everything combined. The two things that usually break that promise are releases and traffic spikes, so I built the platform layer to handle both automatically. New code goes out to 5% of traffic first and rolls itself back in under a minute if error rates or latency move the wrong way. Capacity scales on two levels — pods in seconds, machines in minutes — across three node pools that keep background work away from customer traffic. The point was that no human has to be awake for any of it.”

The 2-minute version

Add the structure. Cover these five beats, in order:

- **The constraint.** 99.95% is 22 minutes a month for everything. A manual rollback takes 10–15 minutes, so a single incident handled by hand costs most of the month's budget. That arithmetic forces automation — it is not a preference.
- **The two threats.** Systems break at releases and at spikes. Almost every design decision addresses one of them.
- **Releases → progressive delivery.** Canary at 5%, then 25%, then 50%, with automated analysis of error rate and p95 latency against the live stable baseline at each step. Any failed check aborts and reverses in under 60 seconds, without a human. It works because the stable pods are never touched — rolling back is a routing change, not a redeploy.
- **Spikes → two-tier autoscaling.** The HPA adds pods in seconds; the cluster autoscaler adds nodes in minutes, 12 to 30. The gap between them is covered by running at a 65% CPU target rather than 90%. Three node pools keep the platform's own tooling and background batch work off the machines serving live traffic.
- **The invisible parts that made it durable.** Three replicas per service spread across three fault domains, health probes that distinguish ‘not ready for traffic’ from ‘restart me,’ PodDisruptionBudgets so routine maintenance cannot take a service to zero, and graceful shutdown so scale-down does not generate errors.

Questions you should expect, and the honest answers

“Why not just aim for 99.99%?”

Because each nine costs disproportionately more. Going to 99.99% means a 4.4-minute monthly budget, which typically requires multi-region deployment, active-active data replication and a much stricter dependency posture. 99.95% was the point where the cost of the next nine exceeded its value for this product — and being able to say that explicitly is more convincing than claiming a bigger number.

“What is the weakest point in this design?”

Stateful dependencies. Everything described here makes the **stateless** layer resilient, and that is genuinely most of the system. But a single managed database remains a shared dependency, and a slow database is felt by every service at once. Mitigations — connection pooling, read replicas, deferring non-critical writes to a queue, readiness probes

that pull pods from rotation rather than restarting them — reduce the impact but do not eliminate it. The honest answer is that the failure mode is degradation rather than outage.

“Did the canary analysis ever produce false positives?”

Yes, and it is worth knowing why. Comparing against a fixed latency threshold produces false aborts whenever the whole system is having a slow moment, and comparing across too short a window produces noise on low-traffic services. Both are fixed the same way: compare the canary against the **live stable version** rather than a constant, and hold each step long enough to gather a statistically meaningful sample. A canary that cries wolf gets disabled by frustrated teams, which is far worse than having no canary at all.

“What would you do differently?”

Introduce the platform primitives earlier. Retrofitting probes, resource requests and rollout templates onto services that already exist means 25 separate negotiations with 25 teams. Making them the default in a shared base chart from the beginning means every new service is born correct, and the standard never has to be argued for again.